



CENTROWEG

TÉCNICO DE CIBERSISTEMAS PARA AUTOMAÇÃO

MA - 78

MATHEUS SCHERER JARDIM

ANA CLARA RODRIGUES ABREU

LUIZA DA SILVA

PROJETO DE CONTROLE DE NÍVEIS

JARAGUÁ DO SUL

2026



MA - 78

Sistemas de Níveis

Sistemas embarcados e
Microcontroladores
SENAI - CENTRO WEG.

Professor: Renan Cesar Tramontini



RESUMO

Este trabalho apresenta o desenvolvimento de um sistema de monitoramento e controle de um reservatório de água utilizando a plataforma Arduino. O objetivo principal do projeto é criar um sistema de controle de nível e consumo para automação, visando a eficiência hídrica. A metodologia incluiu a utilização de cinco sensores digitais para detectar o nível físico da água (de 15% a 85%) com lógica de cascata, um Arduino Uno para o processamento, e um potenciômetro para simular a variação final do volume. Para a interface e atuação, utilizou-se um servo motor para representar a bomba, um display LCD para exibição de dados em tempo real e memória EEPROM para salvamento não-volátil do histórico de consumo. O software foi desenvolvido em linguagem C++ e versionado via Git/GitHub, utilizando o simulador Wokwi para validação prévia do circuito. Os resultados demonstraram um monitoramento preciso e segurança automatizada na detecção de falhas de sensores. Conclui-se que o sistema é uma solução eficiente e de baixo custo, garantindo a integridade dos dados e o controle estável do nível do reservatório.

Palavras-chave: Arduino. Automação. Controle de Nível. C++.



ABSTRACT

This work presents the development of a water reservoir monitoring and control system using the Arduino platform. The main objective of the project is to create a level and consumption control system for automation, aiming at water efficiency. The methodology included the use of five digital sensors to detect the physical water level (from 15% to 85%) with cascade logic, an Arduino Uno for processing, and a potentiometer to simulate the final volume variation. For the interface and actuation, a servo motor was used to represent the pump, an LCD display for real-time data display, and EEPROM memory for non-volatile saving of consumption history. The software was developed in C++ and versioned via Git/GitHub, using the Wokwi simulator for prior circuit validation. The results demonstrated precise monitoring and automated safety in detecting sensor failures. It is concluded that the system is an efficient and low-cost solution, guaranteeing data integrity and stable control of the reservoir level.

Keywords: Arduino. Automation. Level Control. C++.



SUMÁRIO

RESUMO.....	3
ABSTRACT.....	4
1. INTRODUÇÃO.....	6
2. DESENVOLVIMENTO.....	6
2.1 Fundamentação Teórica.....	6
2.2 Funcionamento Do Hardware.....	6
2.3 Descrição Do Protótipo E Materiais.....	7
2.3.1 Protótipos Anteriores.....	8
2.4 Arquitetura Do Software E Lógica De Controle.....	9
2.4.1 Validação De Dados E Segurança.....	9
2.4.2 Monitoramento De Volume E Consumo.....	9
2.4.3 Persistência Em Memória Não Volátil.....	9
2.5. Funcionamento Do Protótipo.....	10
2.6. Ferramentas De Suporte.....	10
2.7. Bibliotecas Utilizadas.....	11
2.8 Registros.....	11
2.8.1 Fotos Do Circuito.....	11
2.8.2 Versionamento.....	13
3. CONCLUSÃO.....	15
4. REFERÊNCIAS.....	16



1. INTRODUÇÃO

Este trabalho tem como objetivo o desenvolvimento de um sistema de controle de nível da água, para evitar a falta ou o transbordamento em um reservatório. Para isso, o foco é o controle da bomba d'água por meio das leituras do sensor de vazão e dos sensores de nível do reservatório.

2. DESENVOLVIMENTO

2.1 Fundamentação Teórica

O projeto fundamenta-se nos conceitos de automação e monitoramento de fluidos. A utilização do microcontrolador Arduino Uno permite a integração de sensores e atuadores no sistema. O controle de nível é essencial para evitar o transbordamento e garantir o suprimento hídrico, enquanto a medição de consumo auxilia na gestão sustentável dos recursos.

2.2 Funcionamento Do Hardware

O Arduino atua como o controlador central, gerenciando a energia através dos pinos de 5V e GND (levados às trilhas laterais da protoboard) e processando os sinais de entrada e saída. O display utiliza um módulo **I2C** (identificado pelos quatro pinos: GND, VCC, SDA, SCL). Isso simplifica muito a fiação (reduz o número de fios):

- **VCC e GND:** Conectados às trilhas de alimentação da protoboard.
- **SDA (Dados):** Conectado ao pino analógico **A4** do Arduino.
- **SCL (Clock):** Conectado ao pino analógico **A5** do Arduino.
 - Nota: no Arduino Uno, as entradas A4 e A5 são padrões para comunicação I2C.

O servo motor é usado para realizar movimentos precisos (geralmente de 0° a 180°). as conexões são: fio marrom/preto GND que vai para o terra (Trilho negativo), fio Vermelho



VCC vai para o trilho positivo (5V) e o Laranja/Amarelo (Sinal) é conectado ao pino digital 8 do Arduino (um pino PWM, essencial para controlar a posição do servo).

O sistema de entrada e interação do projeto é composto por dois tipos de componentes manuais que permitem o controle dinâmico do dispositivo. Primeiramente, foram instalados cinco **Slide switch** na protoboard, conectados aos pinos digitais **6, 7, 9, 10 e 11** do Arduino Uno. Há também um **potenciômetro (sensor rotativo)** posicionado centralmente na protoboard. Suas extremidades estão conectadas às trilhas de alimentação de **5V** e **GND**, enquanto o pino central (cursor) está vinculado à porta analógica **A0**. Essa configuração permite que o Arduino realize a leitura de valores variáveis de tensão, possibilitando um controle analógico preciso, como o ajuste gradual da vazão da água.

Tudo isso está sendo conectado por uma protoboard.

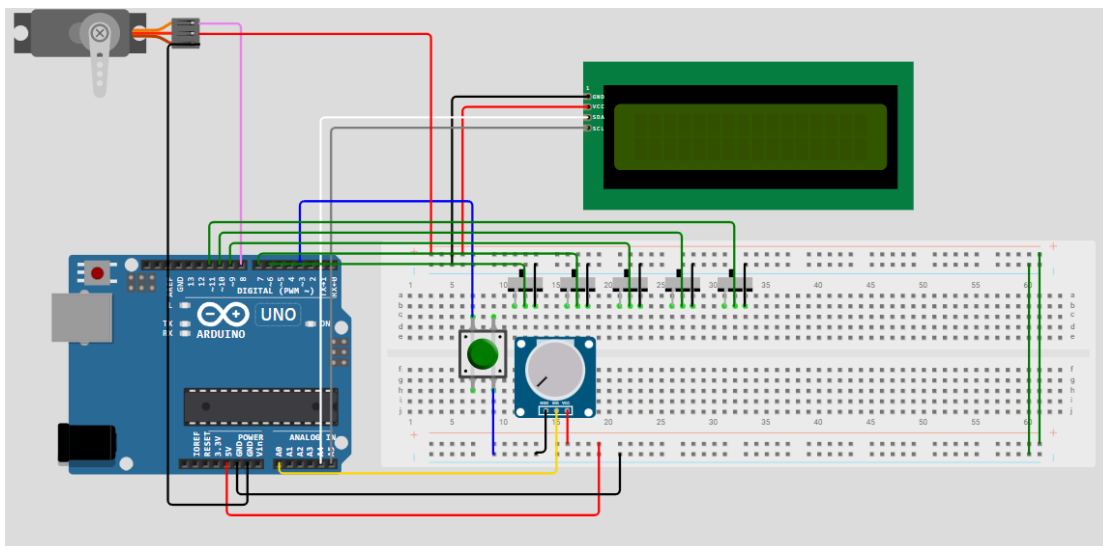
2.3 Descrição Do Protótipo E Materiais

A construção do protótipo utilizou componentes eletrônicos presentes no material disponibilizado nas aulas

- **Microcontrolador:** Arduino Uno (processamento de sinais);
- **Sensores de Nível:** Cinco interruptores digitais (*Micro Switches*) que operam em lógica binária (0 ou 1);
- **Interface Visual:** Display LCD 16x2 com protocolo de comunicação I2C;
- **Atuador de Bomba:** Servo Motor (simulação da válvula ou contator da bomba);
- **Sensor Analógico:** Potenciômetro para ajuste de volume e simulação de consumo.

2.3.1 Protótipos Anteriores

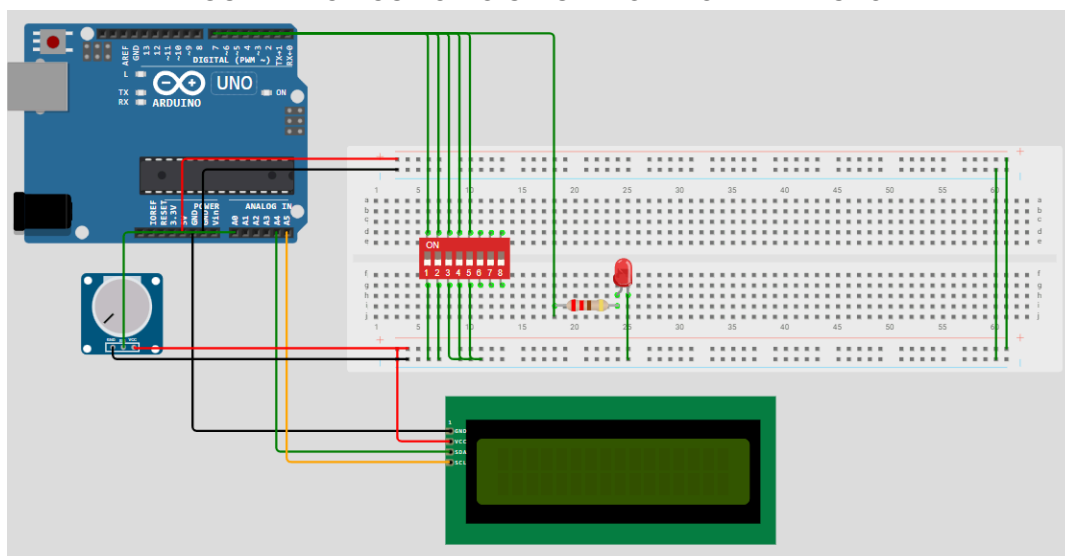
FIGURA 1 - CIRCUITO NO SIMULADOR WOKWI VERSÃO 1



FONTE: Acervo próprio (2026)

Link do projeto Versão 1: <https://wokwi.com/projects/458325265993349121>

FIGURA 2 - CIRCUITO NO SIMULADOR WOKWI VERSÃO 2



FONTE: Acervo próprio (2026)

Link do projeto Versão 2: <https://wokwi.com/projects/458769913226566657>



2.4 Arquitetura Do Software E Lógica De Controle

O software foi desenvolvido em linguagem C++, utilizando a IDE oficial do Arduino. A lógica de programação foi dividida em três pilares fundamentais:

2.4.1 Validação De Dados E Segurança

Para mitigar falhas físicas nos sensores, o algoritmo implementa uma **Cascata de Validação**. Um sensor de nível superior só é considerado "válido" se todos os sensores inferiores a ele também estiverem ativos. Caso contrário, o sistema identifica uma incoerência (ex: sensor de 60% ativo com o de 30% inativo), entra em estado de erro e interrompe o funcionamento da bomba por segurança.

2.4.2 Monitoramento De Volume E Consumo

O sistema utiliza uma variável flutuante (*float*) para calcular o volume atual em litros (de 0 a 8500 L). O consumo total é contabilizado sempre que a leitura atual do volume for menor que a leitura anterior, registrando o histórico de gastos do usuário.

2.4.3 Persistência Em Memória Não Volátil

Devido à natureza volátil da memória RAM, utilizou-se a biblioteca EEPROM.h para gravar os dados de consumo e volume a cada 5 segundos. Este procedimento garante que, em situações de queda de energia ou reinicialização do sistema, os dados históricos não serão perdidos, retomando o monitoramento do ponto exato da interrupção. A EEPROM tem uma vida útil limitada. No Arduino, a EEPROM é especificada para suportar 100.000 ciclos de escrita/apagamento por posição. No entanto, as leituras



são ilimitadas. Isso significa que você pode ler da EEPROM quantas vezes quiser sem comprometer sua vida útil.

2.5. Funcionamento Do Protótipo

O fluxo de operação inicia-se com uma rotina de inicialização de 5 segundos. Após esse período, o sistema entra em um ciclo contínuo:

1. **Varredura:** Leitura dos estados dos sensores e do potenciômetro.
2. **Verificação de Erro:** Validação da integridade dos sensores.
3. **Controle do Atuador:** O servo motor é posicionado em 180° (Bomba Ligada) se o nível estiver abaixo do limite de 85%, e retorna a 90° (Bomba Desligada) ao atingir a capacidade máxima ou detectar falhas.
4. **Atualização de Interface:** Exibição dos dados de porcentagem, volume absoluto e consumo total no LCD.

2.6. Ferramentas De Suporte

1. **Wokwi:** Utilizado para simulação do circuito.
2. **Trello:** Utilizado para organização (Método Kanban).
3. **GitHub:** Utilizado para versionar e armazenar o código.
4. **Gmail:** Utilizado para comunicação e trocar informações.
5. **Canva:** Utilizado para criar a apresentação em slides.

Trello: <https://trello.com/b/NZMxZ0ES/bomba-de-agua>

GitHub: https://github.com/anaclara389-glitch/bomba_de_agua

Link do wokwi: <https://wokwi.com/projects/458949761161382913>

Apresentação do Canva:

https://www.canva.com/design/DAHEPnHRNio/vmSPvR8-czCJgi6-WrHnEQ/edit?utm_content=DAHEPnHRNio&utm_campaign=designshare&utm_medium=link2&utm_source=sharebutton



2.7. Bibliotecas Utilizadas

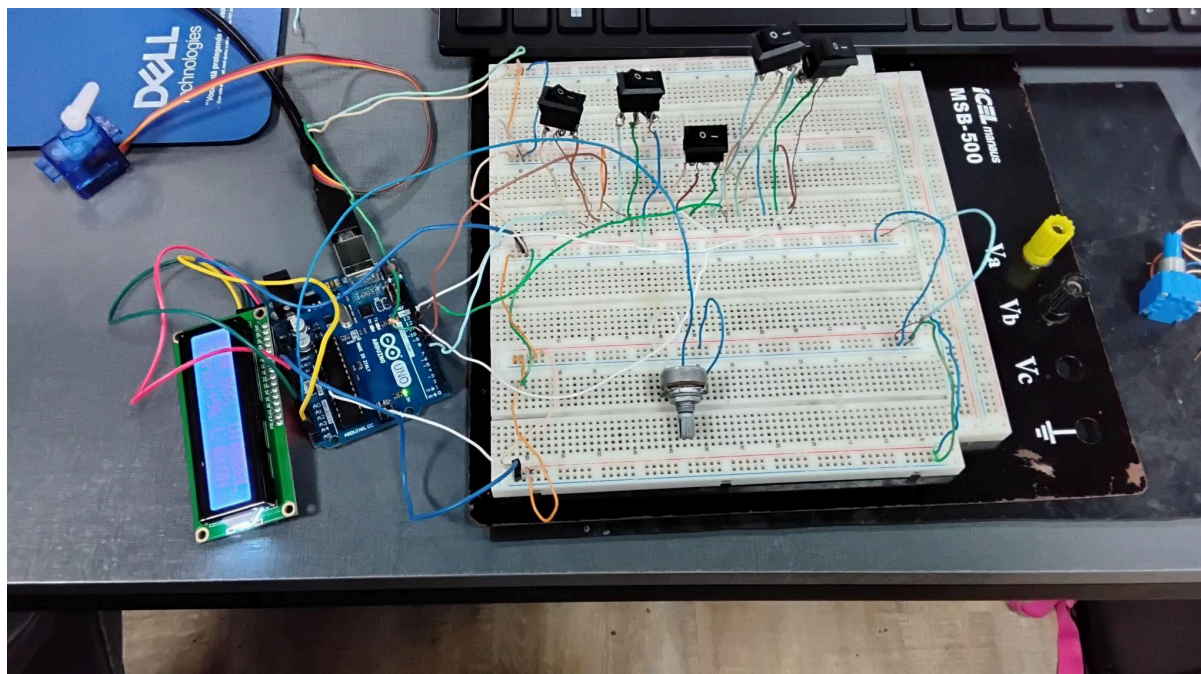
- **WIRE:** A biblioteca **Wire.h** é uma das mais fundamentais no ecossistema Arduino (C/C++). Ela serve para permitir a comunicação entre o microcontrolador e outros dispositivos utilizando o protocolo **I2C** (Inter-Integrated Circuit).
- **LiquidCrystal_I2C:** A biblioteca **LiquidCrystal_I2C.h** é uma extensão da biblioteca padrão de LCD do Arduino, desenvolvida especificamente para controlar displays de cristal líquido que possuem um **adaptador I2C** soldado na parte traseira.
- **EEPROM:** A biblioteca **EEPROM.h** é a ferramenta do Arduino que permite ler e escrever dados na **Memória Não Volátil** do microcontrolador (o "disco rígido" interno do chip).
- **Servo:** A biblioteca **Servo.h** é a ferramenta padrão do Arduino para controlar **servomotores**. Ela simplifica drasticamente a programação, pois lida com a geração dos pulsos elétricos complexos (chamados de PWM - *Pulse Width Modulation*).

2.8 Registros

2.8.1 Fotos Do Circuito

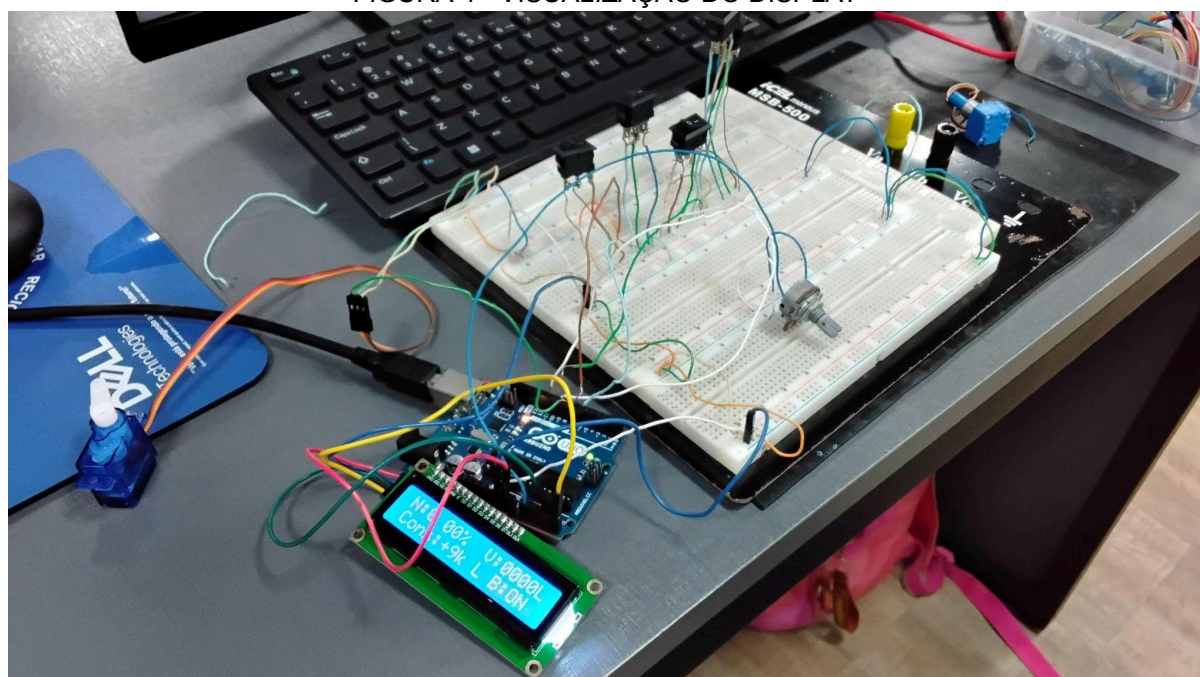
As fotos do circuito montado, em formato *.jpg*, podem ser visualizadas a seguir, demonstrando a disposição física dos componentes. A Figura 3 apresenta o detalhe da protoboard central, evidenciando as chaves de controle e a densidade das conexões. A Figura 4 oferece uma visão abrangente dos periféricos principais, como o microcontrolador Arduino Uno, o display LCD com interface I2C e o micro servo motor. O registro fotográfico permite uma visualização detalhada da prototipagem e da fiação, essencial para a replicação do sistema.

FIGURA 3 - CIRCUITO FÍSICO



FONTE: Acervo próprio (2026)

FIGURA 4 - VISUALIZAÇÃO DO DISPLAY



FONTE: Acervo próprio (2026)



2.8.2 Versionamento

Visando otimizar o desenvolvimento, facilitar a manutenção e garantir uma colaboração eficiente, o software do projeto foi modularizado diretamente na IDE do Arduino. O uso de múltiplas abas (arquivos *.ino*) permitiu separar as responsabilidades lógicas de cada componente do sistema. Essa abordagem orientada a módulos torna o código mais limpo e facilita o versionamento integrado via Git e GitHub, minimizando conflitos de *merge* e edição entre os membros da equipe.

A arquitetura do código foi estruturada em quatro arquivos principais:

1. Código Base (Main): É o arquivo principal do projeto, responsável por centralizar o fluxo de execução. Nele encontram-se as funções obrigatórias do Arduino (*setup()* e *loop()*). O código base atua como o maestro do sistema, importando os demais módulos, inicializando a lógica de tempo (como a varredura a cada 5 segundos) e orquestrando a comunicação entre os sensores, o display e a bomba d'água de forma contínua.

2. Módulo da Bomba (Servo Motor): Este arquivo isola todas as funções e rotinas referentes ao acionamento e parada do atuador físico. Nele, a biblioteca *Servo.h* é instanciada e são definidos os parâmetros de controle de posição (ex: 180° para simular a bomba ligada e 90° para bomba desligada). Essa separação permite que a lógica de bloqueio de segurança e acionamento da bomba seja ajustada sem interferir na leitura dos sensores.

3. Módulo do Display: Arquivo dedicado exclusivamente à interface visual e comunicação I2C utilizando a biblioteca *LiquidCrystal_I2C.h*. Ele agrupa as funções responsáveis por formatar e imprimir os dados na tela LCD 16x2. Ao isolar este módulo, a atualização em tempo real das variáveis de porcentagem de nível, volume absoluto em litros e o consumo histórico (gravado na EEPROM) ocorre de maneira fluida e organizada.



4. Módulo dos Sensores: Este arquivo concentra a lógica de leitura e validação das entradas físicas do sistema. Nele está contida a rotina de varredura dos interruptores e do potenciômetro (simulador de vazão). É neste módulo que a "Cascata de Validação" é executada, garantindo que o estado dos sensores seja coerente (evitando, por exemplo, que um nível de 60% seja lido como verdadeiro se o de 30% estiver inativo) antes de enviar os dados finais para o *Código Base*.

A divisão do código nesses quatro pilares não apenas tornou o encapsulamento das funções mais robusto, como também otimizou o registro de histórico de *commits* no repositório do projeto, permitindo rastrear exatamente qual componente sofreu alterações ao longo do tempo.

O histórico detalhado de todas as versões do projeto, abrangendo os códigos-fonte e esquemas elétricos, encontra-se documentado e disponível para consulta através de um link para um documento do Google Docs. O documento de versionamento (Acervo próprio, 2026) pode ser acessado no seguinte endereço:

https://docs.google.com/document/d/1IIMNS14px8wjlsjedd3g2N6dN4yyknLXNNWP_EgNj3A/edit?tab=t.0. A documentação de versionamento foi fundamental para rastrear o desenvolvimento, facilitar a colaboração e garantir a reprodutibilidade dos resultados alcançados.



3. CONCLUSÃO

A execução deste projeto permitiu o desenvolvimento de um sistema eficiente e seguro para o controle de nível e monitoramento de reservatórios hídricos. Através da integração entre o microcontrolador Arduino Uno e os componentes periféricos, foi possível atingir o objetivo principal de automatizar o acionamento da bomba d'água, mitigando riscos de transbordamento ou desabastecimento.

Um dos pontos de maior relevância no desenvolvimento foi a implementação da lógica de cascata para validação dos sensores. Essa estratégia demonstrou-se fundamental para aumentar a confiabilidade do protótipo, garantindo que falhas físicas nos interruptores de nível não resultem em leituras incoerentes ou acionamentos indevidos da bomba. Além disso, a simulação do volume e o cálculo do consumo acumulado via software apresentaram uma solução viável para a gestão consciente do recurso hídrico, transformando dados brutos de sensores em informações úteis ao usuário final por meio da interface LCD.

A utilização da memória EEPROM para a persistência de dados solucionou o problema da volatilidade das informações em casos de interrupção de energia, embora a limitação de ciclos de escrita da mesma tenha sido devidamente considerada no código por meio de intervalos de salvamento.

Conclui-se, portanto, que o protótipo cumpre as exigências técnicas propostas para um sistema de automação hídrico.



4. REFERÊNCIAS

ARDUINO. **EEPROM**. Arduino Documentation, 2022. Disponível em: <https://docs.arduino.cc/learn/built-in-libraries/eeprom/>. Acesso em: 19 mar. 2026.

ARDUINO. **Wire**. Arduino Documentation, 2026. Disponível em: <https://docs.arduino.cc/language-reference/pt/en/functions/communication/wire/>. Acesso em: 26 mar. 2026.

BRABANDER, Frank de. **LiquidCrystal_I2C**. Arduino Documentation, 2016. Disponível em: <https://docs.arduino.cc/libraries/liquidcrystal-i2c/>. Acesso em: 26 mar. 2026.

LABORATÓRIO de Eletrônica. **Como Funciona uma Ponte H?**. YouTube, 2024. Disponível em: https://www.youtube.com/shorts/d5_yqWQus4Q. Acesso em: 19 mar. 2026.

MARGOLIS, Michael. **Servo**. Arduino Documentation, 2025. Disponível em: <https://docs.arduino.cc/libraries/servo/>. Acesso em: 26 mar. 2026.

SANTOS, Rui; SANTOS, Sara. **Arduino EEPROM Explained: Remember Last LED State**. Random Nerd Tutorials, 2021. Disponível em: <https://randomnerdtutorials.com/arduino-eeprom-explained-remember-last-led-state/>. Acesso em: 17 mar. 2024.